

# EN3160 - Image Processing Assignment 1

**Name:** Hasith Nettikumara

**Index Number:** 230436X

**GitHub Repository:**

<https://github.com/hasith2003/EN3160-Intensity-Transformations-Filtering>

(WHEN DIRECTLY COPYING AND PASTING THIS LINK, THERE MIGHT BE A SPACE INSIDE THE WORD "github", so remove that)

This notebook contains the implementations and results for Assignment 1 on intensity transformations and neighborhood filtering.

```
In [ ]: %pip install matplotlib opencv-python numpy
```

```
In [2]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
IMAGE_DIR = Path("images")
```

```
In [3]: def show_image(image, title="", cmap=None, figsize=(6, 6)):
plt.figure(figsize=figsize)

    if image.ndim == 3:
        # OpenCV loads color images in BGR format
        image_rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
        plt.imshow(image_rgb)
    else:
        plt.imshow(image, cmap=cmap if cmap else "gray")

    plt.title(title)
    plt.axis("off")
    plt.show()
```

## Question 1 - Piecewise Linear Intensity Transformation

A piecewise linear intensity transformation is applied to the given grayscale image using a set of specified breakpoints. Linear interpolation is used to determine the output intensity values between consecutive breakpoints.

```
In [139... q1_portrait_file = IMAGE_DIR / "q1.jpeg"

q1_portrait_gray = cv.imread(str(q1_portrait_file),cv.IMREAD_GRAYSCALE)
```

```
if q1_portrait_gray is None:
    raise FileNotFoundError(f"Could not load Question 1 image: {q1_portrait_file}")
```

```
In [140... q1_breakpoints_reference = np.array([[0, 0],[50, 50],[100, 150],[150, 255],[255,
q1_breakpoints_trial_a = np.array([[0, 0],[40, 30],[90, 110],[150, 210],[210, 2
q1_breakpoints_trial_b = np.array([[0, 0],[50, 25],[100, 125],[160, 220],[220, 2
q1_breakpoints_trial_c = np.array([[0, 0],[40, 30],[90, 100],[150, 175],[210, 2
q1_breakpoints_trial_d = np.array([[0, 0],[40, 28],[90, 98],[150, 168],[210, 22
q1_breakpoints_trial_e = np.array([[0, 0],[40, 32],[90, 102],[150, 170],[210, 2
```

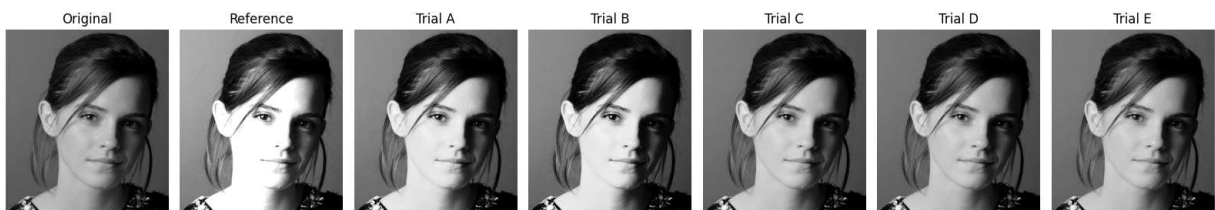
```
In [141... def intensity_transform(im,bp):
    return np.clip(np.interp(im.astype(float),bp[:,0],bp[:,1]),0,255).astype(np.uin
```

```
In [144... q1_breakpoints = [q1_breakpoints_reference,q1_breakpoints_trial_a,q1_breakpoints_tr
q1_outputs = [intensity_transform(q1_portrait_gray,bp) for bp in q1_breakpoints]
```

```
In [145... q1_trial_images = [q1_portrait_gray] + q1_outputs
q1_trial_titles = ["Original","Reference","Trial A","Trial B","Trial C","Trial D","

fig,axes = plt.subplots(1,7,figsize=(16,3))
for ax,img,title in zip(axes,q1_trial_images,q1_trial_titles):
    ax.imshow(img,cmap="gray",vmin=0,vmax=255)
    ax.set_title(title)
    ax.axis("off")

plt.tight_layout()
plt.show()
```



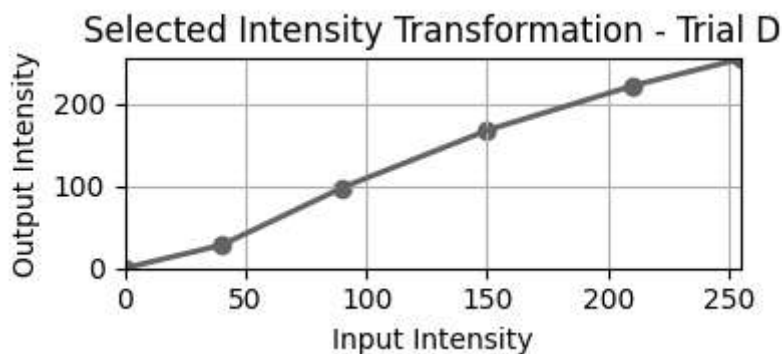
```
In [9]: # Select Trial D as the final intensity transformation
q1_selected_breakpoints = q1_breakpoints_trial_d.copy()
q1_selected_output = intensity_transform(q1_portrait_gray,q1_selected_breakpoints)
```

```
In [147... q1_final_figure, q1_final_axes = plt.subplots(1,2,figsize=(5, 2))
# Original image
q1_final_axes[0].imshow(q1_portrait_gray,cmap="gray",vmin=0,vmax=255)
q1_final_axes[0].set_title("Original Image")
q1_final_axes[0].axis("off")
# Selected transformed image - Trial D
q1_final_axes[1].imshow(q1_selected_output,cmap="gray",vmin=0,vmax=255)
q1_final_axes[1].set_title("Selected Transformed Image (Trial D)")
q1_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```

## Original Image Selected Transformed Image (Trial D)



```
In [123]: q1_selected_intensity_axis = np.arange(256)
q1_selected_mapping = np.interp(q1_selected_intensity_axis, q1_selected_breakpoints[
plt.figure(figsize=(4, 2))
plt.plot(q1_selected_intensity_axis, q1_selected_mapping, linewidth=2)
plt.scatter(q1_selected_breakpoints[:, 0], q1_selected_breakpoints[:, 1])
plt.xlabel("Input Intensity")
plt.ylabel("Output Intensity")
plt.title("Selected Intensity Transformation - Trial D")
plt.xlim(0, 255)
plt.ylim(0, 255)
plt.grid(True)
plt.tight_layout()
plt.show()
```



## Question 2 - White Matter and Gray Matter Enhancement

Piecewise linear intensity transformations are used to separately accentuate white matter and gray matter in the proton-density brain image.

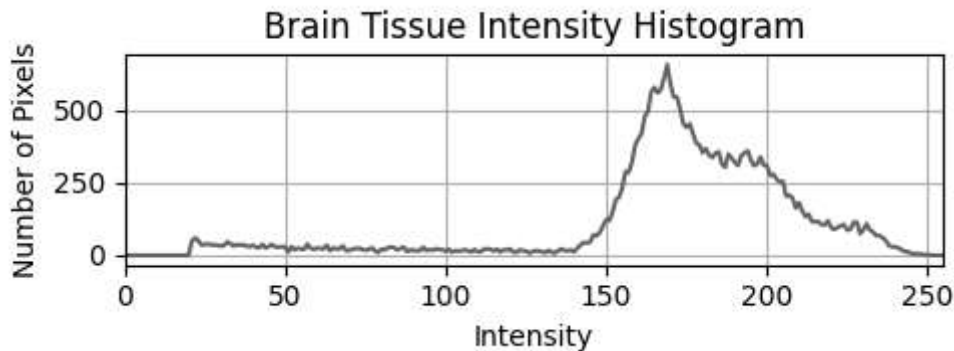
The image intensity distribution is first examined to identify suitable intensity ranges for the two tissue types. Separate transformations are then designed for white matter and gray matter enhancement.

```
In [12]: q2_brain_file = IMAGE_DIR / "q2.jpeg"

q2_brain_gray = cv.imread(str(q2_brain_file), cv.IMREAD_GRAYSCALE)
```

```
if q2_brain_gray is None:
    raise FileNotFoundError(f"Could not load Question 2 image: {q2_brain_file}")
```

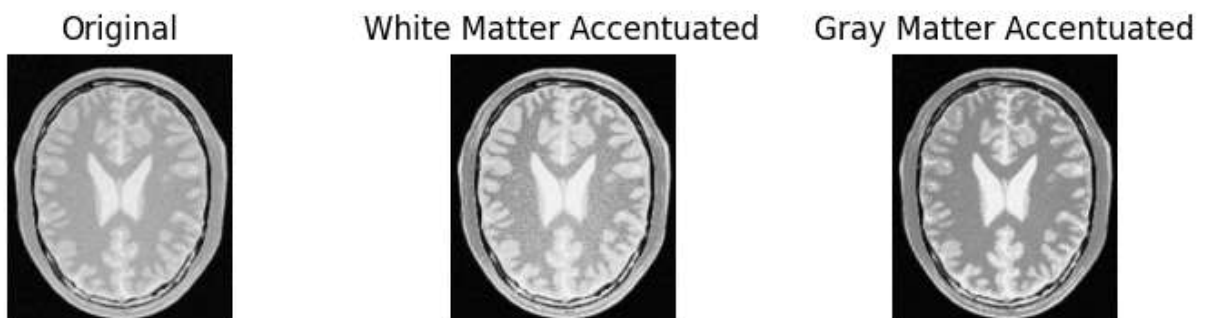
```
In [116... q2_tissue_pixels = q2_brain_gray[q2_brain_gray > 20]
q2_tissue_histogram,_ = np.histogram(q2_tissue_pixels,bins=256,range=(0,256))
plt.figure(figsize=(5,2))
plt.plot(np.arange(256),q2_tissue_histogram)
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.title("Brain Tissue Intensity Histogram")
plt.xlim(0,255)
plt.grid(True)
plt.tight_layout()
plt.show()
```



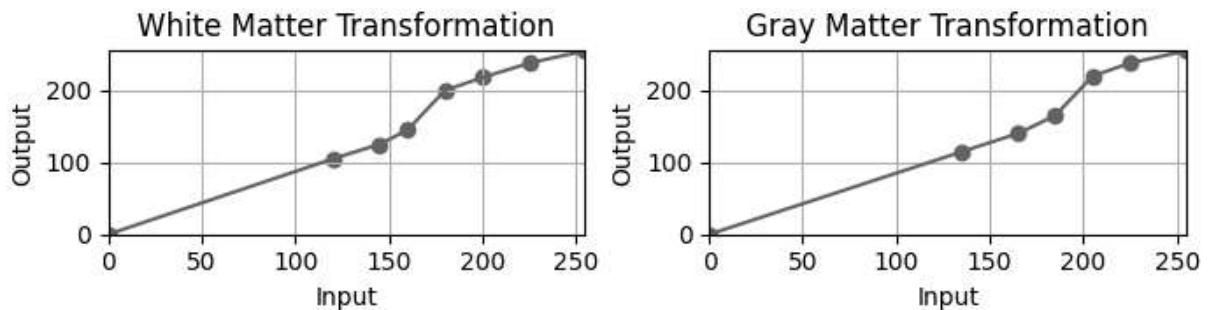
```
In [14]: # Final piecewise-linear transformation for white matter enhancement
q2_white_matter_breakpoints = np.array([[0,0],[120,105],[145,125],[160,145],[180,205]]
# Final piecewise-linear transformation for gray matter enhancement
q2_gray_matter_breakpoints = np.array([[0,0],[135,115],[165,140],[185,165],[205,220]
```

```
In [15]: q2_white_matter_output = intensity_transform(q2_brain_gray,q2_white_matter_breakpoi
q2_gray_matter_output = intensity_transform(q2_brain_gray,q2_gray_matter_breakpoint
```

```
In [156... q2_final_figure,q2_final_axes = plt.subplots(1,3,figsize=(8,2))
q2_final_images = [q2_brain_gray,q2_white_matter_output,q2_gray_matter_output]
q2_final_titles = ["Original","White Matter Accentuated","Gray Matter Accentuated"]
for q2_final_index in range(3):
    q2_final_axes[q2_final_index].imshow(q2_final_images[q2_final_index],cmap="gray"
    q2_final_axes[q2_final_index].set_title(q2_final_titles[q2_final_index])
    q2_final_axes[q2_final_index].axis("off")
plt.tight_layout()
plt.show()
```



```
In [157... x = np.arange(256)
fig,ax = plt.subplots(1,2,figsize=(7,2))
for a,bp,title in zip(ax,[q2_white_matter_breakpoints,q2_gray_matter_breakpoints],
a.plot(x,np.interp(x,bp[:,0],bp[:,1]))
a.scatter(bp[:,0],bp[:,1])
a.set_title(title+" Transformation")
a.set_xlabel("Input")
a.set_ylabel("Output")
a.set_xlim(0,255)
a.set_ylim(0,255)
a.grid()
plt.tight_layout()
plt.show()
```



## Discussion

The histogram was used to identify approximate tissue-intensity ranges. Separate piecewise-linear transformations enhance contrast in the selected white- and gray-matter regions while preserving the overall anatomical appearance.

## Question 3 - Gamma Correction

Gamma correction is applied to the L channel in  $L^*a^*b^*$  space. Several  $\gamma$  values are tested and the original and corrected histograms are compared.

```
In [19]: q3_image_path = IMAGE_DIR/"q3.jpeg"
q3_image_bgr = cv.imread(str(q3_image_path))

if q3_image_bgr is None:
    raise FileNotFoundError(f"Could not load: {q3_image_path}")

q3_image_rgb = cv.cvtColor(q3_image_bgr,cv.COLOR_BGR2RGB)

q3_image_lab = cv.cvtColor(q3_image_bgr,cv.COLOR_BGR2LAB)
q3_lightness_channel,q3_a_channel,q3_b_channel = cv.split(q3_image_lab)
```

```
In [20]: def q3_gamma_correct_lightness(q3_lightness_input,q3_gamma_value):
q3_normalized_lightness = q3_lightness_input.astype(np.float32)/255.0
q3_gamma_corrected_normalized = np.power(q3_normalized_lightness,q3_gamma_value)
```

```
q3_gamma_corrected_lightness = np.clip(q3_gamma_corrected_normalized*255.0,0,255)
return q3_gamma_corrected_lightness.astype(np.uint8)
```

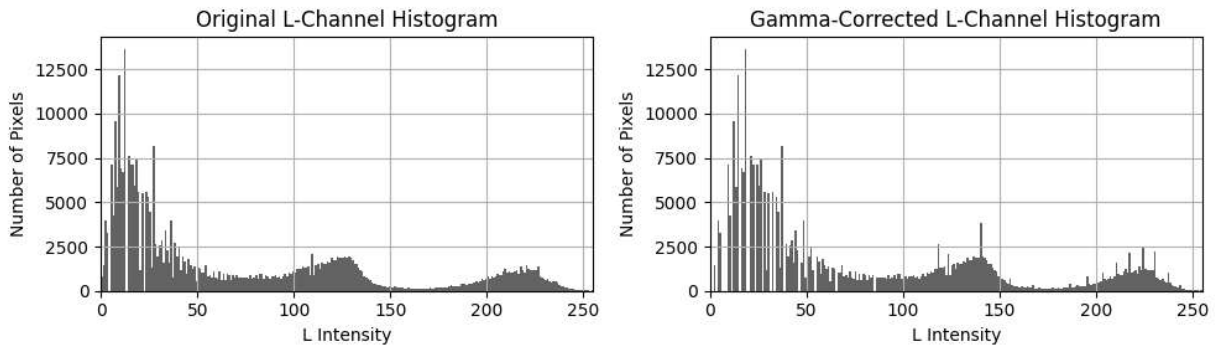
```
In [21]: q3_gamma_candidates = [0.5,0.7,0.85,1.0]
q3_gamma_outputs_rgb = []
for q3_gamma_candidate in q3_gamma_candidates:
    q3_corrected_lightness = q3_gamma_correct_lightness(q3_lightness_channel,q3_gamma_candidate)
    q3_corrected_lab = cv.merge([q3_corrected_lightness,q3_a_channel,q3_b_channel])
    q3_corrected_bgr = cv.cvtColor(q3_corrected_lab,cv.COLOR_LAB2BGR)
    q3_corrected_rgb = cv.cvtColor(q3_corrected_bgr,cv.COLOR_BGR2RGB)
    q3_gamma_outputs_rgb.append(q3_corrected_rgb)
```

```
In [22]: q3_gamma_figure,q3_gamma_axes = plt.subplots(1,5,figsize=(22,5))
# Original
q3_gamma_axes[0].imshow(q3_image_rgb)
q3_gamma_axes[0].set_title("Original")
q3_gamma_axes[0].axis("off")
# Gamma-corrected versions
for q3_gamma_index,q3_gamma_candidate in enumerate(q3_gamma_candidates):
    q3_gamma_axes[q3_gamma_index+1].imshow(q3_gamma_outputs_rgb[q3_gamma_index])
    q3_gamma_axes[q3_gamma_index+1].set_title(f"Gamma = {q3_gamma_candidate}")
    q3_gamma_axes[q3_gamma_index+1].axis("off")
plt.tight_layout()
plt.show()
```



```
In [23]: q3_selected_gamma = 0.85
q3_selected_lightness = q3_gamma_correct_lightness(q3_lightness_channel,q3_selected_gamma)
q3_selected_lab = cv.merge([q3_selected_lightness,q3_a_channel,q3_b_channel])
q3_selected_bgr = cv.cvtColor(q3_selected_lab,cv.COLOR_LAB2BGR)
q3_selected_rgb = cv.cvtColor(q3_selected_bgr,cv.COLOR_BGR2RGB)
```

```
In [24]: q3_histogram_figure,q3_histogram_axes = plt.subplots(1,2,figsize=(10,3))
# Original L-channel histogram
q3_histogram_axes[0].hist(q3_lightness_channel.ravel(),bins=256,range=(0,256))
q3_histogram_axes[0].set_title("Original L-Channel Histogram")
q3_histogram_axes[0].set_xlabel("L Intensity")
q3_histogram_axes[0].set_ylabel("Number of Pixels")
q3_histogram_axes[0].set_xlim(0,255)
q3_histogram_axes[0].grid(True)
# Gamma-corrected L-channel histogram
q3_histogram_axes[1].hist(q3_selected_lightness.ravel(),bins=256,range=(0,256))
q3_histogram_axes[1].set_title("Gamma-Corrected L-Channel Histogram")
q3_histogram_axes[1].set_xlabel("L Intensity")
q3_histogram_axes[1].set_ylabel("Number of Pixels")
q3_histogram_axes[1].set_xlim(0,255)
q3_histogram_axes[1].grid(True)
plt.tight_layout()
plt.show()
```



## Discussion

$\gamma = 0.85$  was selected because it reveals shadow detail without excessive brightening. Since  $\gamma < 1$ , the corrected L-channel histogram shifts toward higher intensities while the a and b channels remain unchanged.

## Question 4 - Vibrance Enhancement

The HSV saturation channel is transformed using the given nonlinear function, with different values of (a) tested to select a visually pleasing result.

```
In [25]: q4_image_path = IMAGE_DIR/"q4.jpeg"
q4_image_bgr = cv.imread(str(q4_image_path))

if q4_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 4 image: {q4_image_path}")

q4_image_rgb = cv.cvtColor(q4_image_bgr,cv.COLOR_BGR2RGB)
```

```
In [26]: q4_image_hsv = cv.cvtColor(q4_image_bgr,cv.COLOR_BGR2HSV)
q4_hue_channel,q4_saturation_channel,q4_value_channel = cv.split(q4_image_hsv)
```

```
In [52]: q4_hsv_figure,q4_hsv_axes = plt.subplots(1,3,figsize=(12,3))
q4_hsv_channels = [q4_hue_channel,q4_saturation_channel,q4_value_channel]
q4_hsv_titles = ["Hue Channel","Saturation Channel","Value Channel"]
for q4_channel_index in range(3):
    q4_hsv_axes[q4_channel_index].imshow(q4_hsv_channels[q4_channel_index],cmap="gr")
    q4_hsv_axes[q4_channel_index].set_title(q4_hsv_titles[q4_channel_index])
    q4_hsv_axes[q4_channel_index].axis("off")
plt.tight_layout()
plt.show()
```



```
In [28]: def q4_apply_vibrance(q4_saturation_input,q4_a_value,q4_sigma_value=70):
    q4_saturation_float = q4_saturation_input.astype(np.float32)
    q4_vibrance_boost = q4_a_value*128.0*np.exp(-((q4_saturation_float-128.0)**2)/(
    q4_enhanced_saturation = np.minimum(q4_saturation_float+q4_vibrance_boost,255.0
    return q4_enhanced_saturation.astype(np.uint8)
```

Testing for multiple values of a

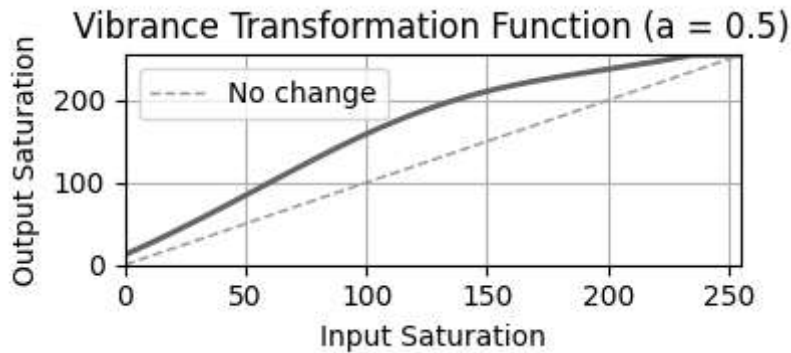
```
In [29]: q4_a_candidates = [0.25,0.50,0.75,1.00]
    q4_vibrance_outputs_rgb = []
    for q4_a_candidate in q4_a_candidates:
        q4_corrected_saturation = q4_apply_vibrance(q4_saturation_channel,q4_a_candidate)
        q4_corrected_hsv = cv.merge([q4_hue_channel,q4_corrected_saturation,q4_value_channel])
        q4_corrected_bgr = cv.cvtColor(q4_corrected_hsv,cv.COLOR_HSV2BGR)
        q4_corrected_rgb = cv.cvtColor(q4_corrected_bgr,cv.COLOR_BGR2RGB)
        q4_vibrance_outputs_rgb.append(q4_corrected_rgb)
```

```
In [30]: q4_compare_figure,q4_compare_axes = plt.subplots(1,5,figsize=(22,5))
    q4_compare_axes[0].imshow(q4_image_rgb)
    q4_compare_axes[0].set_title("Original")
    q4_compare_axes[0].axis("off")
    for q4_candidate_index,q4_a_candidate in enumerate(q4_a_candidates):
        q4_compare_axes[q4_candidate_index+1].imshow(q4_vibrance_outputs_rgb[q4_candidate_index])
        q4_compare_axes[q4_candidate_index+1].set_title(f"a = {q4_a_candidate}")
        q4_compare_axes[q4_candidate_index+1].axis("off")
    plt.tight_layout()
    plt.show()
```



```
In [31]: q4_selected_a = 0.50
    q4_selected_saturation = q4_apply_vibrance(q4_saturation_channel,q4_selected_a)
    q4_selected_hsv = cv.merge([q4_hue_channel,q4_selected_saturation,q4_value_channel])
    q4_selected_bgr = cv.cvtColor(q4_selected_hsv,cv.COLOR_HSV2BGR)
    q4_selected_rgb = cv.cvtColor(q4_selected_bgr,cv.COLOR_BGR2RGB)
```

```
In [113... q4_intensity_axis = np.arange(256,dtype=np.float32)
    q4_transformation_curve = np.minimum(q4_intensity_axis+q4_selected_a*128.0*np.exp(-
    plt.figure(figsize=(4,2))
    plt.plot(q4_intensity_axis,q4_transformation_curve,linewidth=2)
    plt.plot([0,255],[0,255],"--",linewidth=1,label="No change")
    plt.xlabel("Input Saturation")
    plt.ylabel("Output Saturation")
    plt.title(f"Vibrance Transformation Function (a = {q4_selected_a})")
    plt.xlim(0,255)
    plt.ylim(0,255)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()
```



## Discussion

Increasing (a) enhances saturation and makes the colors more vivid, while excessively large values appear unnatural. Therefore, (a = 0.5) was chosen as the best balance between vibrance and natural appearance.

## Question 5 - Histogram Equalization

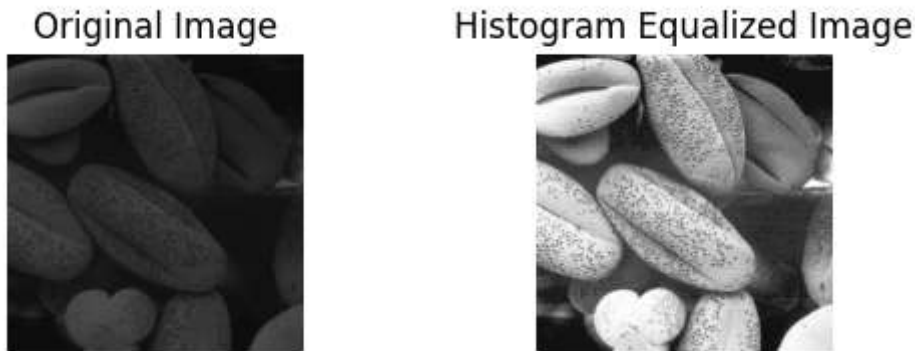
Histogram equalization is implemented manually using the histogram and CDF, and the results are compared before and after equalization.

```
In [33]: q5_image_path = IMAGE_DIR / "q5.jpeg"
q5_image_gray = cv.imread(str(q5_image_path),cv.IMREAD_GRAYSCALE)
if q5_image_gray is None:
    raise FileNotFoundError(f"Could not load Question 5 image: {q5_image_path}")

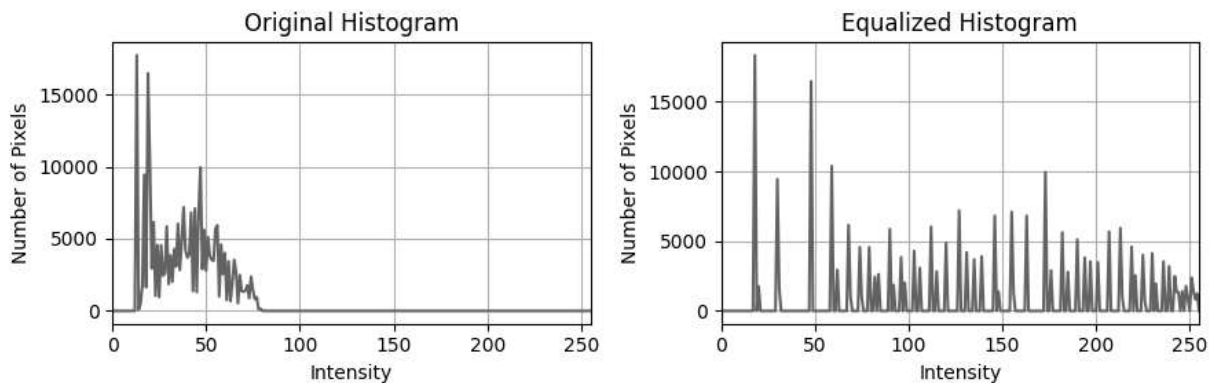
In [34]: def q5_histogram_equalize(q5_input_image):
q5_histogram = np.bincount(q5_input_image.ravel(),minlength=256)
q5_probability_distribution = q5_histogram/q5_input_image.size
q5_cdf = np.cumsum(q5_probability_distribution)
q5_mapping = np.floor(255*q5_cdf).astype(np.uint8)
q5_equalized_image = q5_mapping[q5_input_image]
return q5_equalized_image,q5_histogram,q5_cdf,q5_mapping

In [35]: q5_equalized_gray,q5_original_histogram,q5_original_cdf,q5_equalization_mapping = q

In [55]: q5_image_figure,q5_image_axes = plt.subplots(1,2,figsize=(6,2))
q5_image_axes[0].imshow(q5_image_gray,cmap="gray",vmin=0,vmax=255)
q5_image_axes[0].set_title("Original Image")
q5_image_axes[0].axis("off")
q5_image_axes[1].imshow(q5_equalized_gray,cmap="gray",vmin=0,vmax=255)
q5_image_axes[1].set_title("Histogram Equalized Image")
q5_image_axes[1].axis("off")
plt.tight_layout()
plt.show()
```



```
In [37]: q5_equalized_histogram = np.bincount(q5_equalized_gray.ravel(),minlength=256)
q5_histogram_figure,q5_histogram_axes = plt.subplots(1,2,figsize=(9,3))
q5_histogram_axes[0].plot(np.arange(256),q5_original_histogram)
q5_histogram_axes[0].set_title("Original Histogram")
q5_histogram_axes[0].set_xlabel("Intensity")
q5_histogram_axes[0].set_ylabel("Number of Pixels")
q5_histogram_axes[0].set_xlim(0,255)
q5_histogram_axes[0].grid(True)
q5_histogram_axes[1].plot(np.arange(256),q5_equalized_histogram)
q5_histogram_axes[1].set_title("Equalized Histogram")
q5_histogram_axes[1].set_xlabel("Intensity")
q5_histogram_axes[1].set_ylabel("Number of Pixels")
q5_histogram_axes[1].set_xlim(0,255)
q5_histogram_axes[1].grid(True)
plt.tight_layout()
plt.show()
```



## Discussion

Histogram equalization spreads the initially concentrated intensities across a wider range, improving contrast and revealing darker details. The resulting histogram is not perfectly uniform because image intensities are discrete.

## Question 6 - Histogram Equalization of the Foreground

The saturation plane is thresholded to extract the foreground, and histogram equalization is applied only to its value channel while preserving the background.

```
In [38]: q6_image_path = IMAGE_DIR / "q6.jpeg"
q6_image_bgr = cv.imread(str(q6_image_path))
if q6_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 6 image: {q6_image_path}")
q6_image_rgb = cv.cvtColor(q6_image_bgr, cv.COLOR_BGR2RGB)
```

```
In [39]: q6_image_hsv = cv.cvtColor(q6_image_bgr, cv.COLOR_BGR2HSV)
q6_hue_channel, q6_saturation_channel, q6_value_channel = cv.split(q6_image_hsv)
```

```
In [40]: q6_hsv_figure, q6_hsv_axes = plt.subplots(1, 3, figsize=(15, 5))
q6_hsv_channels = [q6_hue_channel, q6_saturation_channel, q6_value_channel]
q6_hsv_titles = ["Hue Plane", "Saturation Plane", "Value Plane"]
for q6_plane_index in range(3):
    q6_hsv_axes[q6_plane_index].imshow(q6_hsv_channels[q6_plane_index], cmap="gray")
    q6_hsv_axes[q6_plane_index].set_title(q6_hsv_titles[q6_plane_index])
    q6_hsv_axes[q6_plane_index].axis("off")
plt.tight_layout()
plt.show()
```



```
In [56]: q6_selected_threshold = 30
_, q6_foreground_mask = cv.threshold(q6_saturation_channel, q6_selected_threshold, 255)
plt.figure(figsize=(5, 2))
plt.imshow(q6_foreground_mask, cmap="gray", vmin=0, vmax=255)
plt.title(f"Initial Foreground Mask (T = {q6_selected_threshold})")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Initial Foreground Mask (T = 30)



```
In [43]: from scipy.ndimage import label
q6_inverse_mask = q6_foreground_mask==0
q6_labels, q6_num_labels = label(q6_inverse_mask)
```

```

q6_refined_mask = q6_foreground_mask.copy()
q6_max_hole_size = 3000
for q6_label_index in range(1,q6_num_labels+1):
    q6_region = q6_labels==q6_label_index
    q6_region_size = np.sum(q6_region)
    q6_touches_boundary = np.any(q6_region[0,:]) or np.any(q6_region[-1,:]) or np.any(q6_region[:,0]) or np.any(q6_region[:,255])
    if not q6_touches_boundary and q6_region_size<=q6_max_hole_size:
        q6_refined_mask[q6_region] = 255

```

```

In [44]: q6_foreground_bgr = cv.bitwise_and(q6_image_bgr,q6_image_bgr,mask=q6_refined_mask)
q6_foreground_value_pixels = q6_value_channel[q6_refined_mask>0]
print("Number of foreground pixels:",q6_foreground_value_pixels.size)

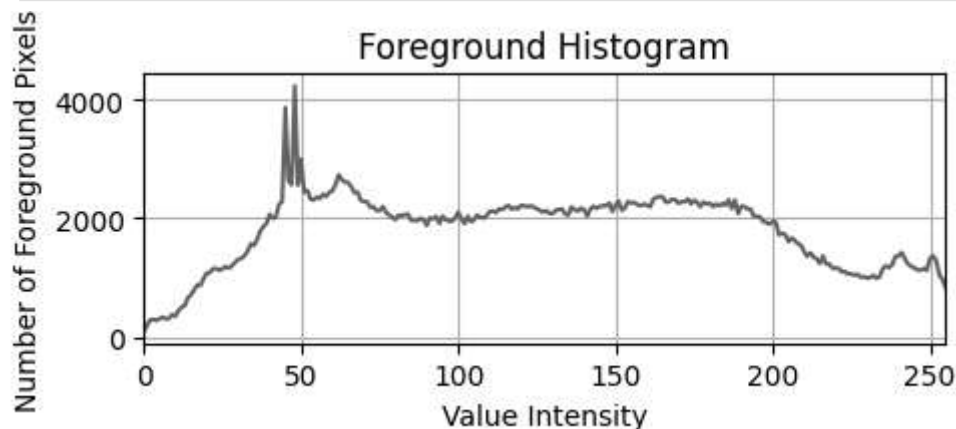
```

Number of foreground pixels: 462937

```

In [108]: q6_foreground_histogram = np.bincount(q6_foreground_value_pixels,minlength=256)
plt.figure(figsize=(5,2.3))
plt.plot(np.arange(256),q6_foreground_histogram)
plt.xlabel("Value Intensity")
plt.ylabel("Number of Foreground Pixels")
plt.title("Foreground Histogram")
plt.xlim(0,255)
plt.grid(True)
plt.tight_layout()
plt.show()

```



```

In [46]: q6_foreground_cdf = np.cumsum(q6_foreground_histogram)

```

```

In [47]: q6_nonzero_cdf = q6_foreground_cdf[q6_foreground_histogram>0]
q6_cdf_min = q6_nonzero_cdf[0]
q6_foreground_pixel_count = q6_foreground_value_pixels.size
q6_equalization_mapping = np.round(255*(q6_foreground_cdf-q6_cdf_min)/(q6_foreground_pixel_count-q6_cdf_min))
q6_equalization_mapping = np.clip(q6_equalization_mapping,0,255).astype(np.uint8)

```

```

In [48]: q6_equalized_value_channel = q6_value_channel.copy()
q6_equalized_value_channel[q6_refined_mask>0] = q6_equalization_mapping[q6_value_channel[q6_refined_mask>0]]

```

```

In [49]: q6_equalized_hsv = cv.merge([q6_hue_channel,q6_saturation_channel,q6_equalized_value_channel])
q6_equalized_bgr = cv.cvtColor(q6_equalized_hsv,cv.COLOR_HSV2BGR)
q6_equalized_rgb = cv.cvtColor(q6_equalized_bgr,cv.COLOR_BGR2RGB)

```

```
In [57]: q6_final_figure, q6_final_axes = plt.subplots(1, 2, figsize=(7, 2))
q6_final_axes[0].imshow(q6_image_rgb)
q6_final_axes[0].set_title("Original Image")
q6_final_axes[0].axis("off")
q6_final_axes[1].imshow(q6_equalized_rgb)
q6_final_axes[1].set_title("Foreground Histogram Equalized")
q6_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```



## Discussion

Equalization spreads the foreground intensities over a wider range, improving its contrast and detail while the background remains unchanged.

## Question 7 – Sobel Filtering

Sobel filtering is performed using OpenCV `filter2D`, manual convolution, and separable 1-D filtering, and the resulting edge maps are compared.

```
In [76]: q7_image_path = IMAGE_DIR / "q7.jpeg"
q7_image_gray = cv.imread(str(q7_image_path), cv.IMREAD_GRAYSCALE)

if q7_image_gray is None:
    raise FileNotFoundError(f"Could not load Question 7 image: {q7_image_path}")
```

```
In [77]: q7_sobel_x_kernel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]], dtype=np.float32)
q7_sobel_y_kernel = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]], dtype=np.float32)
```

```
In [78]: q7_gradient_x_filter2d = cv.filter2D(q7_image_gray.astype(np.float32), cv.CV_32F, q7_
q7_gradient_y_filter2d = cv.filter2D(q7_image_gray.astype(np.float32), cv.CV_32F, q7_
q7_gradient_magnitude_filter2d = np.sqrt(q7_gradient_x_filter2d**2 + q7_gradient_y_fi
q7_gradient_magnitude_filter2d = cv.normalize(q7_gradient_magnitude_filter2d, None, 0
```

```
In [79]: def q7_manual_filter(q7_input_image, q7_kernel):
    q7_kernel_height, q7_kernel_width = q7_kernel.shape
    q7_pad_y = q7_kernel_height // 2
    q7_pad_x = q7_kernel_width // 2
    q7_padded_image = np.pad(q7_input_image.astype(np.float32), ((q7_pad_y, q7_pad_y)
```

```

q7_output_image = np.zeros_like(q7_input_image,dtype=np.float32)
for q7_row in range(q7_input_image.shape[0]):
    for q7_col in range(q7_input_image.shape[1]):
        q7_local_region = q7_padded_image[q7_row:q7_row+q7_kernel_height,q7_col:q7_col+q7_kernel_width]
        q7_output_image[q7_row,q7_col] = np.sum(q7_local_region*q7_kernel)
return q7_output_image

```

```

In [80]: q7_gradient_x_manual = q7_manual_filter(q7_image_gray,q7_sobel_x_kernel)
q7_gradient_y_manual = q7_manual_filter(q7_image_gray,q7_sobel_y_kernel)
q7_gradient_magnitude_manual = np.sqrt(q7_gradient_x_manual**2+q7_gradient_y_manual**2)
q7_gradient_magnitude_manual = cv.normalize(q7_gradient_magnitude_manual,None,0,255)

```

```

In [81]: q7_smoothing_column = np.array([[1],[2],[1]],dtype=np.float32)
q7_derivative_row = np.array([[1,0,-1]],dtype=np.float32)
q7_derivative_column = np.array([[1],[0],[-1]],dtype=np.float32)
q7_smoothing_row = np.array([[1,2,1]],dtype=np.float32)

```

```

In [82]: q7_temp_x = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_smoothing_col)
q7_gradient_x_separable = cv.filter2D(q7_temp_x,cv.CV_32F,q7_derivative_row)
q7_temp_y = cv.filter2D(q7_image_gray.astype(np.float32),cv.CV_32F,q7_smoothing_row)
q7_gradient_y_separable = cv.filter2D(q7_temp_y,cv.CV_32F,q7_derivative_column)
q7_gradient_magnitude_separable = np.sqrt(q7_gradient_x_separable**2+q7_gradient_y_separable**2)
q7_gradient_magnitude_separable = cv.normalize(q7_gradient_magnitude_separable,None,0,255)

```

```

In [83]: q7_compare_figure,q7_compare_axes = plt.subplots(1,4,figsize=(11,3))
q7_compare_axes[0].imshow(q7_image_gray,cmap="gray")
q7_compare_axes[0].set_title("Original")
q7_compare_axes[1].imshow(q7_gradient_magnitude_filter2d,cmap="gray")
q7_compare_axes[1].set_title("filter2D Sobel")
q7_compare_axes[2].imshow(q7_gradient_magnitude_manual,cmap="gray")
q7_compare_axes[2].set_title("Manual Sobel")
q7_compare_axes[3].imshow(q7_gradient_magnitude_separable,cmap="gray")
q7_compare_axes[3].set_title("Separable Sobel")
for q7_axis in q7_compare_axes:
    q7_axis.axis("off")
plt.tight_layout()
plt.show()

```



## Discussion

All three methods produce similar edge maps. The separable implementation performs the same Sobel operation using successive 1-D filters, reducing computation compared with

direct 2-D convolution.

## Question 8 – Image Zooming Using Interpolation

Nearest-neighbor and bilinear interpolation are implemented for  $\times 4$  enlargement and compared with the original images using normalized SSD.

```
In [84]: q8_original_1_bgr = cv.imread(str(IMAGE_DIR/"im01.png"))
q8_small_1_bgr = cv.imread(str(IMAGE_DIR/"im01small.png"))
q8_original_2_bgr = cv.imread(str(IMAGE_DIR/"im02.png"))
q8_small_2_bgr = cv.imread(str(IMAGE_DIR/"im02small.png"))
if any(q8_image is None for q8_image in [q8_original_1_bgr,q8_small_1_bgr,q8_origin
    raise FileNotFoundError("Could not load one or more Question 8 images.")
q8_original_1_rgb = cv.cvtColor(q8_original_1_bgr,cv.COLOR_BGR2RGB)
q8_small_1_rgb = cv.cvtColor(q8_small_1_bgr,cv.COLOR_BGR2RGB)
q8_original_2_rgb = cv.cvtColor(q8_original_2_bgr,cv.COLOR_BGR2RGB)
q8_small_2_rgb = cv.cvtColor(q8_small_2_bgr,cv.COLOR_BGR2RGB)
```

```
In [85]: def q8_zoom_nearest(q8_input_image,q8_scale_factor):
q8_input_height,q8_input_width = q8_input_image.shape[:2]
q8_output_height = int(round(q8_input_height*q8_scale_factor))
q8_output_width = int(round(q8_input_width*q8_scale_factor))
q8_output_image = np.zeros((q8_output_height,q8_output_width,q8_input_image.sha
for q8_output_row in range(q8_output_height):
    for q8_output_col in range(q8_output_width):
        q8_source_row = int(round(q8_output_row/q8_scale_factor))
        q8_source_col = int(round(q8_output_col/q8_scale_factor))
        q8_source_row = min(q8_source_row,q8_input_height-1)
        q8_source_col = min(q8_source_col,q8_input_width-1)
        q8_output_image[q8_output_row,q8_output_col] = q8_input_image[q8_source
return q8_output_image
```

```
In [86]: def q8_zoom_bilinear(q8_input_image,q8_scale_factor):
q8_input_height,q8_input_width = q8_input_image.shape[:2]
q8_output_height = int(round(q8_input_height*q8_scale_factor))
q8_output_width = int(round(q8_input_width*q8_scale_factor))
q8_output_image = np.zeros((q8_output_height,q8_output_width,q8_input_image.sha
for q8_output_row in range(q8_output_height):
    for q8_output_col in range(q8_output_width):
        q8_source_y = q8_output_row/q8_scale_factor
        q8_source_x = q8_output_col/q8_scale_factor
        q8_y0 = int(np.floor(q8_source_y))
        q8_x0 = int(np.floor(q8_source_x))
        q8_y1 = min(q8_y0+1,q8_input_height-1)
        q8_x1 = min(q8_x0+1,q8_input_width-1)
        q8_y0 = min(q8_y0,q8_input_height-1)
        q8_x0 = min(q8_x0,q8_input_width-1)
        q8_dy = q8_source_y-q8_y0
        q8_dx = q8_source_x-q8_x0
        q8_top = (1-q8_dx)*q8_input_image[q8_y0,q8_x0]+q8_dx*q8_input_image[q8_
        q8_bottom = (1-q8_dx)*q8_input_image[q8_y1,q8_x0]+q8_dx*q8_input_image[
```

```
q8_output_image[q8_output_row,q8_output_col] = (1-q8_dy)*q8_top+q8_dy*q8_bottom
return np.clip(q8_output_image,0,255).astype(np.uint8)
```

```
In [87]: q8_scale_factor = 4
q8_image_1_nearest = q8_zoom_nearest(q8_small_1_rgb,q8_scale_factor)
q8_image_1_bilinear = q8_zoom_bilinear(q8_small_1_rgb,q8_scale_factor)
q8_image_2_nearest = q8_zoom_nearest(q8_small_2_rgb,q8_scale_factor)
q8_image_2_bilinear = q8_zoom_bilinear(q8_small_2_rgb,q8_scale_factor)
```

```
In [88]: q8_compare_1_figure,q8_compare_1_axes = plt.subplots(1,3,figsize=(10,3))
q8_compare_1_axes[0].imshow(q8_original_1_rgb)
q8_compare_1_axes[0].set_title("Original Image 1")
q8_compare_1_axes[1].imshow(q8_image_1_nearest)
q8_compare_1_axes[1].set_title("Nearest Neighbor x4")
q8_compare_1_axes[2].imshow(q8_image_1_bilinear)
q8_compare_1_axes[2].set_title("Bilinear x4")
for q8_axis in q8_compare_1_axes:
    q8_axis.axis("off")
plt.tight_layout()
plt.show()
```



```
In [89]: q8_compare_2_figure,q8_compare_2_axes = plt.subplots(1,3,figsize=(10,3))
q8_compare_2_axes[0].imshow(q8_original_2_rgb)
q8_compare_2_axes[0].set_title("Original Image 2")
q8_compare_2_axes[1].imshow(q8_image_2_nearest)
q8_compare_2_axes[1].set_title("Nearest Neighbor x4")
q8_compare_2_axes[2].imshow(q8_image_2_bilinear)
q8_compare_2_axes[2].set_title("Bilinear x4")
for q8_axis in q8_compare_2_axes:
    q8_axis.axis("off")
plt.tight_layout()
plt.show()
```



```
In [90]: def q8_normalized_ssd(q8_reference_image,q8_test_image):
q8_reference_float = q8_reference_image.astype(np.float32)
q8_test_float = q8_test_image.astype(np.float32)
q8_squared_difference = (q8_reference_float-q8_test_float)**2
```

```
q8_normalized_ssd_value = np.mean(q8_squared_difference)/(255.0**2)
return q8_normalized_ssd_value
```

```
In [91]: q8_ssd_1_nearest = q8_normalized_ssd(q8_original_1_rgb,q8_image_1_nearest)
q8_ssd_1_bilinear = q8_normalized_ssd(q8_original_1_rgb,q8_image_1_bilinear)
q8_ssd_2_nearest = q8_normalized_ssd(q8_original_2_rgb,q8_image_2_nearest)
q8_ssd_2_bilinear = q8_normalized_ssd(q8_original_2_rgb,q8_image_2_bilinear)
print("Image 1")
print(f"Nearest-neighbor normalized SSD: {q8_ssd_1_nearest:.6f}")
print(f"Bilinear normalized SSD: {q8_ssd_1_bilinear:.6f}")
print()
print("Image 2")
print(f"Nearest-neighbor normalized SSD: {q8_ssd_2_nearest:.6f}")
print(f"Bilinear normalized SSD: {q8_ssd_2_bilinear:.6f}")
```

```
Image 1
Nearest-neighbor normalized SSD: 0.003926
Bilinear normalized SSD: 0.003080
```

```
Image 2
Nearest-neighbor normalized SSD: 0.000994
Bilinear normalized SSD: 0.000753
```

## Discussion

Nearest-neighbor interpolation produces more block-like artifacts, while bilinear interpolation gives smoother results. Lower normalized SSD indicates a reconstruction closer to the original image.

## Question 9 – Foreground Segmentation and Background Blurring

GrabCut separates the flower from the background, which is strongly blurred and recombined with the sharp foreground.

```
In [92]: q9_image_path = IMAGE_DIR/"q9.jpeg"
q9_image_bgr = cv.imread(str(q9_image_path))
if q9_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 9 image: {q9_image_path}")
q9_image_rgb = cv.cvtColor(q9_image_bgr,cv.COLOR_BGR2RGB)
```

```
In [ ]: q9_height,q9_width = q9_image_bgr.shape[:2]
q9_grabcut_mask = np.zeros((q9_height,q9_width),dtype=np.uint8)
q9_background_model = np.zeros((1,65),dtype=np.float64)
q9_foreground_model = np.zeros((1,65),dtype=np.float64)
q9_rectangle = (int(0.05*q9_width),int(0.05*q9_height),int(0.90*q9_width),int(0.90*
cv.grabCut(q9_image_bgr,q9_grabcut_mask,q9_rectangle,q9_background_model,q9_foregro
```

```
In [94]: q9_binary_mask = np.where((q9_grabcut_mask==cv.GC_FGD)|(q9_grabcut_mask==cv.GC_PR_F
```

```

In [95]: q9_foreground_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_binary_mask)
q9_background_mask = cv.bitwise_not(q9_binary_mask)
q9_background_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_background_mas
q9_foreground_rgb = cv.cvtColor(q9_foreground_bgr,cv.COLOR_BGR2RGB)
q9_background_rgb = cv.cvtColor(q9_background_bgr,cv.COLOR_BGR2RGB)

In [96]: # Remove small false foreground regions by keeping only the largest connected compo
q9_num_labels,q9_labels,q9_stats,q9_centroids = cv.connectedComponentsWithStats(q9_
# Ignore label 0 because it is the background
q9_largest_label = 1+np.argmax(q9_stats[1:,cv.CC_STAT_AREA])
q9_final_mask = np.where(q9_labels==q9_largest_label,255,0).astype(np.uint8)

In [97]: # Extract foreground using cleaned mask
q9_foreground_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_final_mask)
# Extract background
q9_background_mask = cv.bitwise_not(q9_final_mask)
q9_background_bgr = cv.bitwise_and(q9_image_bgr,q9_image_bgr,mask=q9_background_mas
# Convert for Matplotlib
q9_foreground_rgb = cv.cvtColor(q9_foreground_bgr,cv.COLOR_BGR2RGB)
q9_background_rgb = cv.cvtColor(q9_background_bgr,cv.COLOR_BGR2RGB)
# Display final segmentation results
fig,axes = plt.subplots(1,3,figsize=(6,2))
axes[0].imshow(q9_final_mask,cmap="gray")
axes[0].set_title("Final Segmentation Mask")
axes[1].imshow(q9_foreground_rgb)
axes[1].set_title("Foreground")
axes[2].imshow(q9_background_rgb)
axes[2].set_title("Background")
for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.show()

```

Final Segmentation Mask



Foreground



Background



```

In [98]: q9_blurred_background_bgr = cv.GaussianBlur(q9_background_bgr,(41,41),0)
q9_enhanced_bgr = cv.add(q9_foreground_bgr,q9_blurred_background_bgr)
q9_enhanced_rgb = cv.cvtColor(q9_enhanced_bgr,cv.COLOR_BGR2RGB)

In [99]: q9_final_figure,q9_final_axes = plt.subplots(1,2,figsize=(6,2))
q9_final_axes[0].imshow(q9_image_rgb)
q9_final_axes[0].set_title("Original Image")
q9_final_axes[0].axis("off")
q9_final_axes[1].imshow(q9_enhanced_rgb)

```

```
q9_final_axes[1].set_title("Enhanced Image")
q9_final_axes[1].axis("off")
plt.tight_layout()
plt.show()
```

Original Image



Enhanced Image



## Discussion

The dark region near the flower occurs because its removed area is represented by zero-valued pixels in the extracted background. Gaussian filtering averages these with nearby pixels, darkening the region around the boundary.

## Question 10 – Bilateral Filtering

OpenCV and custom bilateral filters are compared with Gaussian smoothing, with quantitative comparison between the bilateral implementations.

```
In [93]: q10_image_path = IMAGE_DIR/"q10.jpeg"
q10_image_bgr = cv.imread(str(q10_image_path))
if q10_image_bgr is None:
    raise FileNotFoundError(f"Could not load Question 10 image: {q10_image_path}")
q10_image_rgb = cv.cvtColor(q10_image_bgr,cv.COLOR_BGR2RGB)
```

```
In [94]: q10_sigma_s = 3.0
q10_sigma_r = 50.0
q10_kernel_size = 9
q10_radius = q10_kernel_size // 2
```

```
In [95]: q10_gaussian_bgr = cv.GaussianBlur(q10_image_bgr,(q10_kernel_size,q10_kernel_size),
q10_opencv_bilateral_bgr = cv.bilateralFilter(q10_image_bgr,d=q10_kernel_size,sigma
q10_gaussian_rgb = cv.cvtColor(q10_gaussian_bgr,cv.COLOR_BGR2RGB)
q10_opencv_bilateral_rgb = cv.cvtColor(q10_opencv_bilateral_bgr,cv.COLOR_BGR2RGB)
```

```
In [96]: def q10_custom_bilateral_filter(q10_input_image,q10_sigma_space,q10_sigma_range,q10
q10_input_float = q10_input_image.astype(np.float32)
q10_height,q10_width = q10_input_float.shape[:2]
q10_padded_image = cv.copyMakeBorder(q10_input_float,q10_filter_radius,q10_filt
q10_coordinate_range = np.arange(-q10_filter_radius,q10_filter_radius+1)
q10_grid_x,q10_grid_y = np.meshgrid(q10_coordinate_range,q10_coordinate_range)
q10_spatial_weights = np.exp(-(q10_grid_x**2+q10_grid_y**2)/(2.0*q10_sigma_spac
```

```

q10_output_image = np.zeros_like(q10_input_float)
for q10_row in range(q10_height):
    for q10_col in range(q10_width):
        q10_patch = q10_padded_image[q10_row:q10_row+2*q10_filter_radius+1,q10_
q10_center_pixel = q10_padded_image[q10_row+q10_filter_radius,q10_col+q
q10_color_difference_squared = np.sum((q10_patch-q10_center_pixel)**2,a
q10_range_weights = np.exp(-q10_color_difference_squared/(2.0*q10_sigma
q10_total_weights = q10_spatial_weights*q10_range_weights
q10_weight_sum = np.sum(q10_total_weights)
q10_output_image[q10_row,q10_col] = np.sum(q10_patch*q10_total_weights[
return np.clip(q10_output_image,0,255).astype(np.uint8)

```

```

In [97]: q10_custom_bilateral_bgr = q10_custom_bilateral_filter(q10_image_bgr,q10_sigma_s,q1
q10_custom_bilateral_rgb = cv.cvtColor(q10_custom_bilateral_bgr,cv.COLOR_BGR2RGB)

```

```

In [100... q10_final_figure,q10_final_axes = plt.subplots(1,4,figsize=(12,3))
q10_final_images = [q10_image_rgb,q10_gaussian_rgb,q10_opencv_bilateral_rgb,q10_cus
q10_final_titles = ["Original","Gaussian","OpenCV Bilateral","Custom Bilateral"]
for q10_index in range(4):
    q10_final_axes[q10_index].imshow(q10_final_images[q10_index])
    q10_final_axes[q10_index].set_title(q10_final_titles[q10_index])
    q10_final_axes[q10_index].axis("off")
plt.tight_layout()
plt.show()

```



```

In [ ]: q10_opencv_float = q10_opencv_bilateral_bgr.astype(np.float64)
q10_custom_float = q10_custom_bilateral_bgr.astype(np.float64)
q10_mse = np.mean((q10_opencv_float-q10_custom_float)**2)
if q10_mse==0:
    q10_psnr = float("inf")
else:
    q10_psnr = 10*np.log10((255.0**2)/q10_mse)
print(f"MSE between OpenCV and custom bilateral: {q10_mse:.4f}")
print(f"PSNR between OpenCV and custom bilateral: {q10_psnr:.2f} dB")

```

MSE between OpenCV and custom bilateral: 15.5296  
 PSNR between OpenCV and custom bilateral: 36.22 dB

## Discussion

Unlike Gaussian filtering, bilateral filtering also considers intensity differences, smoothing similar regions while preserving edges. MSE and PSNR quantify the agreement between the OpenCV and custom implementations.